Artificial intelligence

Logic and planning

Stepas Toliautas
2026-10-12

Formal logic

- Agent's knowledge can be expressed in two ways:
  - *Procedural* knowledge – source code, hardware circuits, muscle memory
  - *Declarative* knowledge – databases, blueprints, books
- Structure of declarative knowledge
  - Units of information (*statements*)
  - Rules for combining and evaluating statements (*logic*)
- Uses of logic
  - To evaluate given statements
  - To formulate new statements (*inference*)

- What do you think of when you hear „logic"?..

Propositional logic

- Similar to logic (*Boolean*) algebra

- Environment model consists of *facts A*, *B*, *C*, …
  - Possible values – *T* (true), *N* (false), *?* (unknown)
- Facts are combined into *statements* using five operators
  - *AND*, *OR*, *NOT*, *EQUIV*, *IMPLIES*

- Statements can be:
  - *Valid* – true for any values of their elements
  - *Satisfiable* – true for some values
  - *Unsatisfiable* – false for any values

Propositional logic: operators

		(also ‘–‘)	(also ∧)	(also ∨)
P	**Q**	**¬P, *NOT***	**P ∩ Q, *AND***	**P ∪ Q, *OR***
N	N	T	N	N
N	T	T	N	T
T	N	N	N	T
T	T	N	T	T

			(also ≡)
P	**Q**	**P → Q, *IMPLIES***	**P ↔ Q, *EQUIV***
N	N	T	T
N	T	T	N
T	N	N	N
T	T	T	T

Propositional logic: exercises

$$(E \vee B) \rightarrow A$$
$$A \rightarrow (\neg J \wedge M)$$
$$B$$

$$E,\ B,\ A,\ J,\ M\ =\ ?$$

	T	N	?
E			V
B	V		
A	V		
J		V	
M	V		

$$P \vee \neg P$$
$$P \wedge \neg P$$
$$P \vee Q \vee (P \equiv Q)$$
$$(P \rightarrow Q) \vee (\neg Q \rightarrow \neg P)$$

	V	P	N
1	V		
2			V
3	V		
4		V	

Are these statements valid?

Limitations of propositional logic

- Do not describe uncertainty
  - Except the very narrow case of missing information
- Do not define objects and properties
  - Concepts of trees, frogs and their greenness broken down to binary facts
- Do not describe grouping of facts
  - The only way of grouping is *conjunction* (*AND*)
- The latter two are problem for humans, but not *computers*
  - Which are a) designed for binary computation and b) really fast
  - Keep that in mind for a few slides

First-order logic

- Environment model contains:
  - Constants (*A*, *B*, *1*, *2*, …)
  - Predicates, or relations (*Above(x,y)*, *Column(x)*, *Sunny()*, …)
  - Functions (*LeftNeighbor(x)*, …)

- Model elements are used to describe *objects*
  - Board square, row number, vacuum cleaner
  - Object may or may not have a named constant;
    for example, squares are usually denoted by row/column pairs

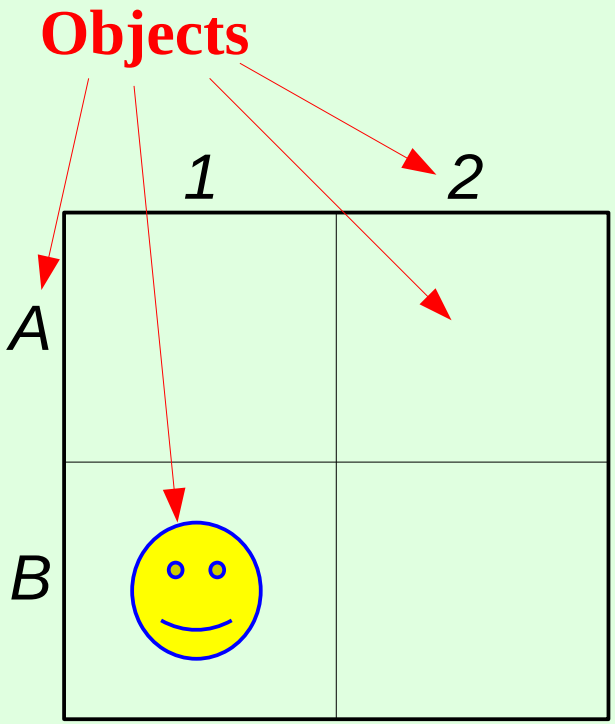
First-order logic: statements

- *Statements* are formulated using:
  - Predicates of constants, variables and functions
  - 0-th order logical expressions using predicates
  - Equality relation
  - Quantifier expressions:
    - „*for all*“ ($\forall x$: …)
    - „*exists*“ ($\exists x$: …)

- Notes:
  - Operator „*for all*“ is commonly used together with *IMPLIES* to define groups of objects
  - Operator „*exists*“ is used to assign a constant property to an object

First-order logic: example

- Constants:
  - *1, 2, A, B, :), >:X ← non-existent monster*
- Predicates:
  - *Square(x,y), Agent(a), Monster(p), Resides(o,x,y)*
- Example statements:
  - „Smiley resides in *B1*“
    - *Resides(:), B, 1)*
  - „There are no monsters here“



- $\forall x, y \; \forall p: \; Square(x, y) \land Monster(p) \rightarrow \neg Resides(p, x, y)$

- Predicates and quantifiers are used on objects, but not on other predicates or functions (thus, „*1st-order*“)

First-order logic: exercise

$$\exists x, y: \; x = y$$

$$\forall x: \; P(x) \wedge \neg P(x)$$

$$(\exists x: x = x) \rightarrow (\forall y \exists z: y = z)$$

$$\exists x: \; P(x)$$

	V	P	N
1	V		
2			V
3	V		
4		V	

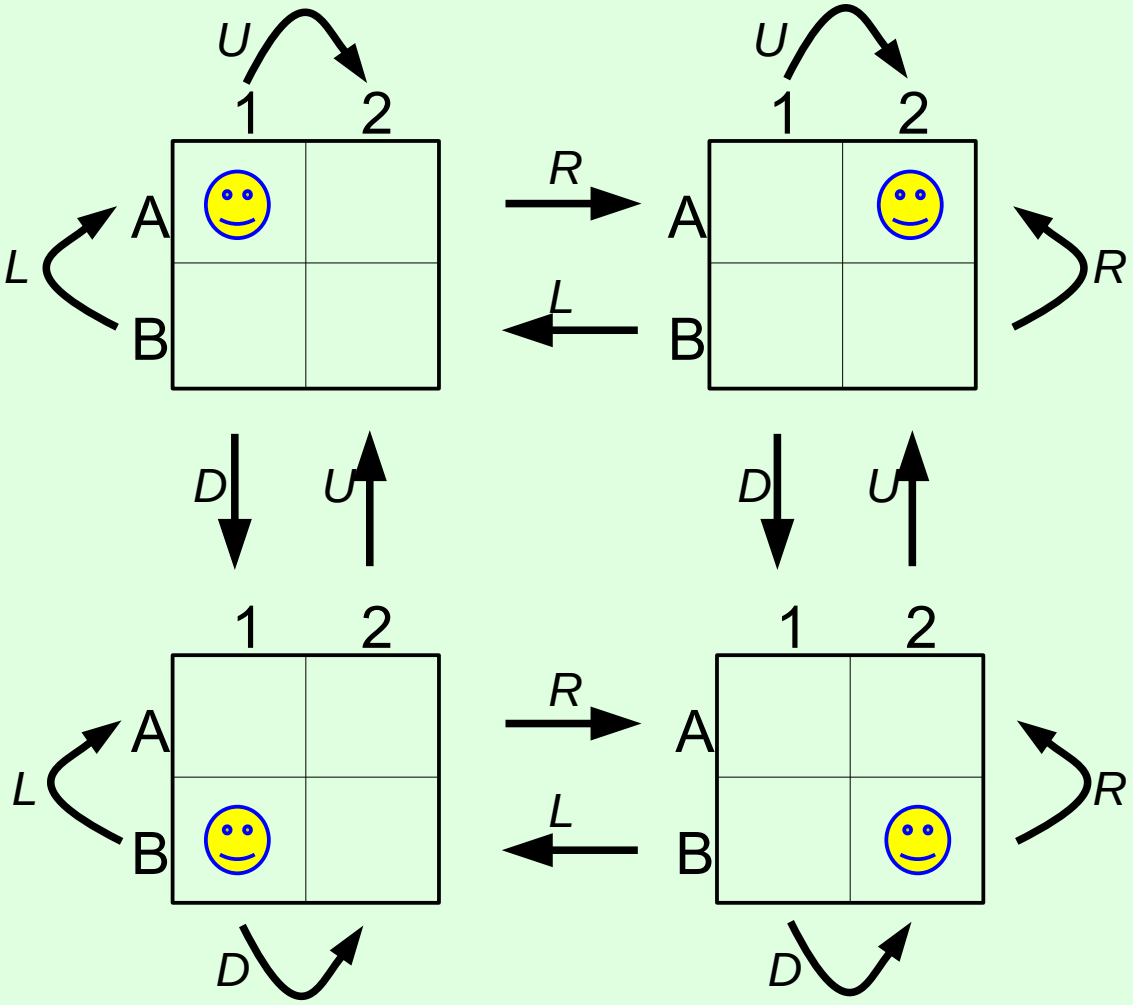
Are these statements valid?

Planning vs. search

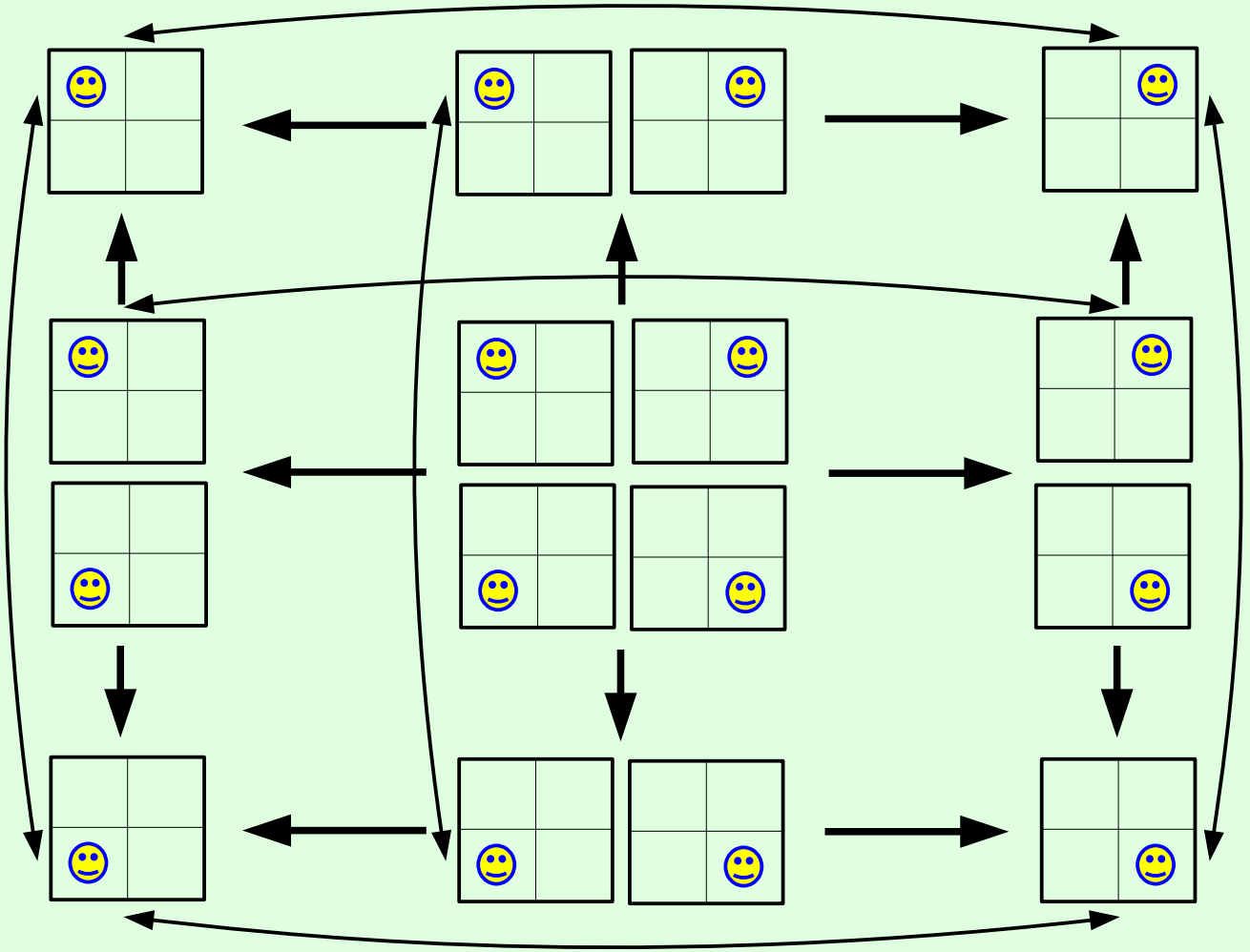
- Environments suitable for searching:
  - Deterministic
  - Fully observable
  - Static
- Solution is a path of (possibly many) steps
- In reality, the optimal solution can change during execution
  - Uncertainty
  - New information
  - Changes in the environment
- The *planning – acting* cycle is more applicable in such cases

- How to treat cases of missing information?

State space



Belief space



Planning in partially observable environments

- Available types of steps – *actions* and *observations*

- In deterministic environments:
  - Actions keep or narrow down the belief state
  - Observations narrow down the belief state

- In stochastic and dynamic environments:
  - Actions ***broaden*** the belief state (increase uncertainty)
  - Observations narrow down the belief state

Planning notation

- Planning step in state space:
  - $s' = Result(s, a)$
  - s – previous state, a – action
  - So a step looks just like successor function!
- Planning step in belief space:
  - $t' = Check(Guess(t, a), i)$
  - t – previous state, a – action, i – observation
- Plans can be described and stored using logic

Classical planning

- State space – k binary $\{T,N\}$ variables
- *Full* (actual) state is defined by the values of all variables
- *Belief* state may consist of:
  - Values of all variables
  - Values of some variables
  - Propositional logic expressions

Classical planning: notation

- Planning problems are commonly *defined* using first-order logic
  - Planning languages *STRIPS*, *ADL* → ***PDDL***
- They can be *solved* using propositional logic
  - A straightforward algorithm can *propositionalize* a first-order problem
- Environment model:
  - *Predicates* and functions
  - *Action* schemas

- ***ACTION*** $Move(p, x_1, y_1, x_2, y_2)$

  Precond: $Agent(p) \wedge Row(x_1) \wedge Col(y_1) \wedge Row(x_2) \wedge Col(y_2) \wedge$
  $\wedge ((Adj(x_1, x_2) \wedge (y_1 = y_2)) \vee (Adj(y_1, y_2) \wedge (x_1 = x_2)))$
  $\wedge Resides(p, x_1, y_1)$

  Effect: $\neg Resides(p, x_1, y_1) \wedge Resides(p, x_2, y_2)$

Classical planning: actions

- Actions are created by passing specific argument values
- Obtained actions may or may not be *applicable* in current state
  - They are applicable if all *preconditions* hold
- New states are generated using action *effects*
  - Negative effects are removed from state description
    - By default, all not-stated values of variables are negative (false)
  - Positive effects are added to the state
  - However, effects cannot add new variables to the description

Classical planning problems

- Problem definition:
  - *Objects* (constants)
  - *Initial* and *goal* states (usually belief states):

  - ***Start*:** $Agent(:)) \wedge Row(A) \wedge Row(B) \wedge Col(1) \wedge Col(2) \wedge$
    $\wedge Resides(:), A, 1) \wedge Adj(...$
    ***Goal*:** $Resides(:), B, 2)$

- Solution is expressed as a *plan*
  - List of actions connecting initial and goal states

- How to find such a plan?

How to find a plan

- Direct search
  - In state space (equal to declarative formulation of search)
  - In belief space (can also treat partially observable problems)
    - Heuristic functions = beliefs with less set variables
- Backward search
  - Search for actions whose effects satisfy goal state/ current state
  - Complements direct search without much complication
- Search in *plan* space
  - Partially sorted action lists are constructed
  - This allows descriptions of alternative sets of plans
    - It usually does not matter which shoe is put first while getting dressed

Chair from IKEA

- Take a look at a simple model
  - Can a goal be reached using available actions?
  - What it will take (e. g. in node expansions) to obtain an answer?
  - Can you think of a way to improve model description?

ACTION $Inspect(n_1, \ldots, n_5, b_1, \ldots, b_5)$

***Precond*:** $Fastened(n_1, b_1) \wedge \ldots \wedge Fastened(n_5, b_5)$

***Effect*:** $Inspected()$

ACTION $Fasten(n, b)$

***Precond*:** $Nut(n) \wedge Bolt(b)$

***Effect*:** $Fastened(n, b) \wedge \neg Nut(n) \wedge \neg Bolt(b)$

***Start*:** $Nut(N_1) \wedge \ldots \wedge Nut(N_4) \wedge Bolt(B_1) \wedge \ldots \wedge Bolt(B_5)$

***Goal*:** $Inspected()$

Classical planning with resources

- *USE* and *CONSUME* work with *resource counts*
  - *USE* temporarily blocks, *CONSUME* removes a resource
- Search tree can now be reduced

$$\begin{aligned}
&\textbf{\textit{ACTION}} && Inspect(n_1,\ldots,n_5,b_1,\ldots,b_5)\\
&\textbf{\textit{Precond}}: && Fastened(n_1,b_1)\wedge\ldots\wedge Fastened(n_5,b_5)\\
&\textbf{\textit{Use}}: && Inspector:1\\
&\textbf{\textit{Effect}}: && Inspected()
\end{aligned}$$

$$\begin{aligned}
&\textbf{\textit{ACTION}} && Fasten(n,b)\\
&\textbf{\textit{Consume}}: && Nuts:1,\ Bolts:1\\
&\textbf{\textit{Effect}}: && Fastened(n,b)
\end{aligned}$$

$$\textbf{\textit{Resources}}: \quad Nuts:4,\ Bolts:5,\ Inspectors:1$$

Painting a chair from IKEA

- So we built a chair...
  - Can a goal be reached using available actions?
  - Are we assured to get an optimal plan?
  - Can you think of a way to improve model description?

ACTION	***Precond*:**	***Effect*:**
$RemoveLid(can)$	$Can(can)$	$Open(can)$
ACTION	***Precond*:**	***Effect*:**
$Paint(x, can)$	$Can(can) \wedge Color(can, c) \wedge \wedge Open(can) \wedge Object(x)$	$Color(x, c) \wedge \neg Can(can)$
ACTION	***Precond*:**	***Effect*:**
$LookAt(x)$	$InView(y) \wedge (x \neq y)$	$InView(x) \wedge \neg InView(y)$

***Start*:** $Object(Table) \wedge Object(Chair) \wedge Can(C_1) \wedge Can(C_2) \wedge InView(Table)$

***Goal*:** $\exists c: \; Color(Chair, c) \wedge Color(Table, c)$

Classical planning with percepts

- *Percepts* update the information used to describe a state
  - Actions may not add new variables
  - So percepts are necessary in most partially-observable cases
    - Ability to obtain knowledge and add it to the model

ACTION	***ACTION***	***ACTION***
$RemoveLid(can)$	$Paint(x, can)$	$LookAt(x)$
PERCEPT	***Precond*:**	
$Color(x, c)$	$Object(x) \wedge InView(x)$	
PERCEPT	***Precond*:**	
$Color(can, c)$	$Can(can) \wedge Open(can) \wedge$ $\wedge\ InView(can)$	

***Start*:** $Object(Table) \wedge Object(Chair) \wedge Can(C_1) \wedge Can(C_2) \wedge InView(Table)$

***Goal*:** $\exists c: Color(Chair, c) \wedge Color(Table, c)$

PDDL software

- planning.domains
- *https://editor.planning.domains/*
  - Open-source PDDL solver including online editor (above)
  - Access via VS Code plugin

- PythonPDDL
- *https://openplan-labs.github.io/PythonPDDL/*
  - Pure *Python* PDDL planning framework
  - Available as a *Python* library and an online workbench (above)
  - Nice search graphs